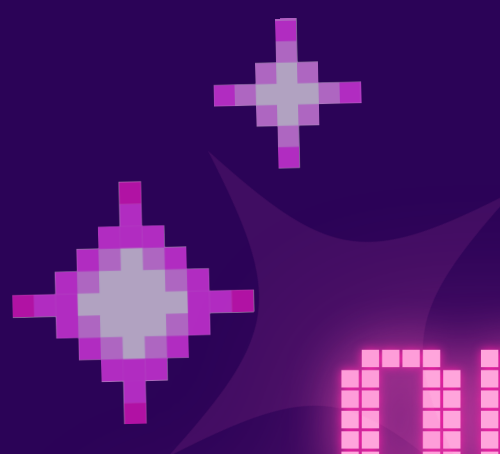




OFICINA DE PROGRAMAÇÃO CALCULADORA COM INTERFACE GRÁFICA





QUEM SÃO AS CODIFICADORAS?

Projeto de extensão da UTFPR-AP destinado a
incentivar mulheres na área de **computação**

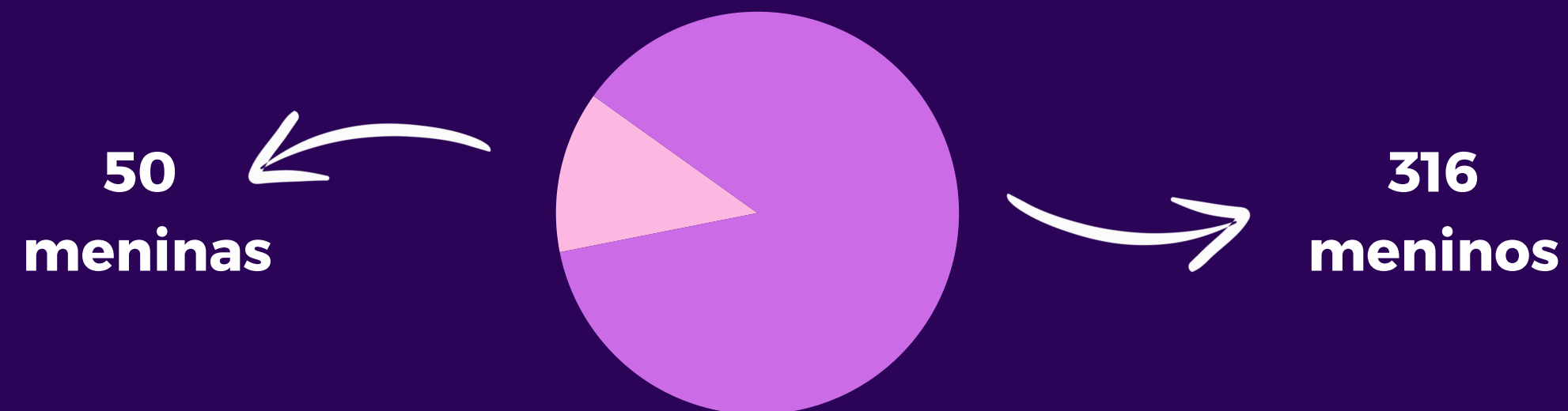




POR QUE ISSO É IMPORTANTE?

No curso de computação do nosso campus,
apenas 13%* dos alunos são mulheres

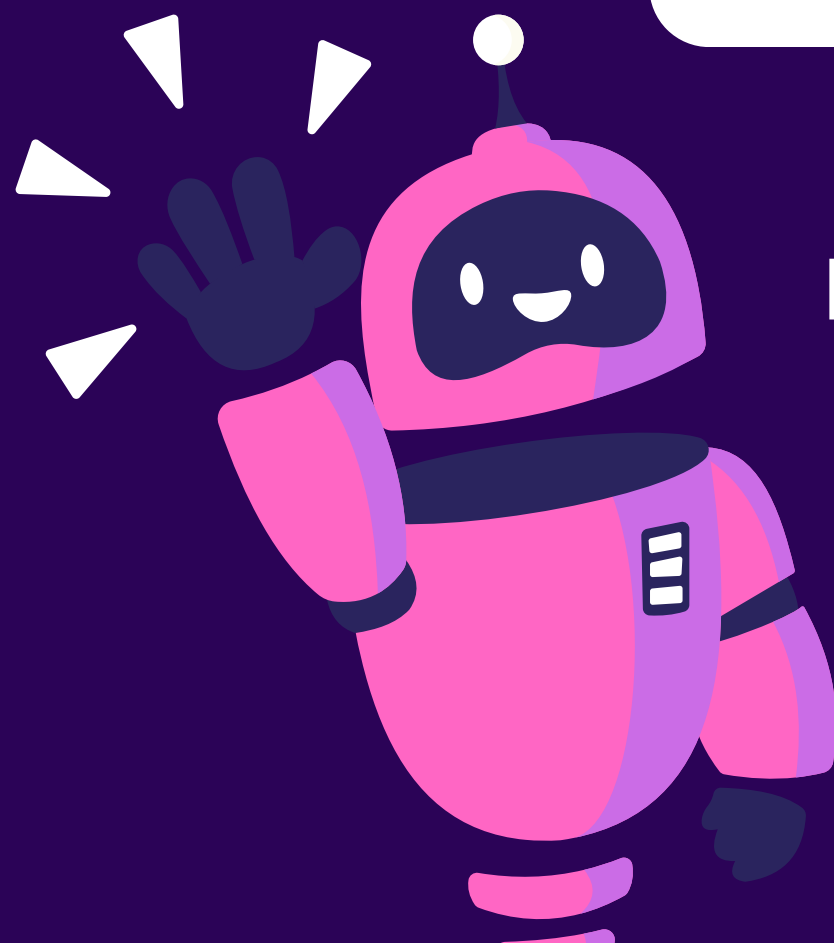
*Dados do Sistema da UTFPR de 2025





COMO PODEMOS MUDAR ISSO?

Computação também é **coisa de menina!**

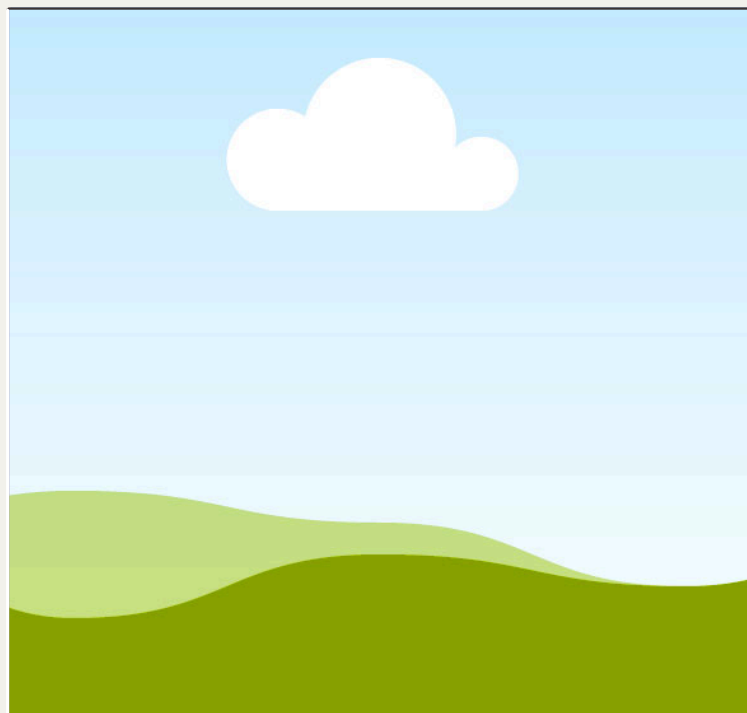


E nós iremos provar isso por meio de:

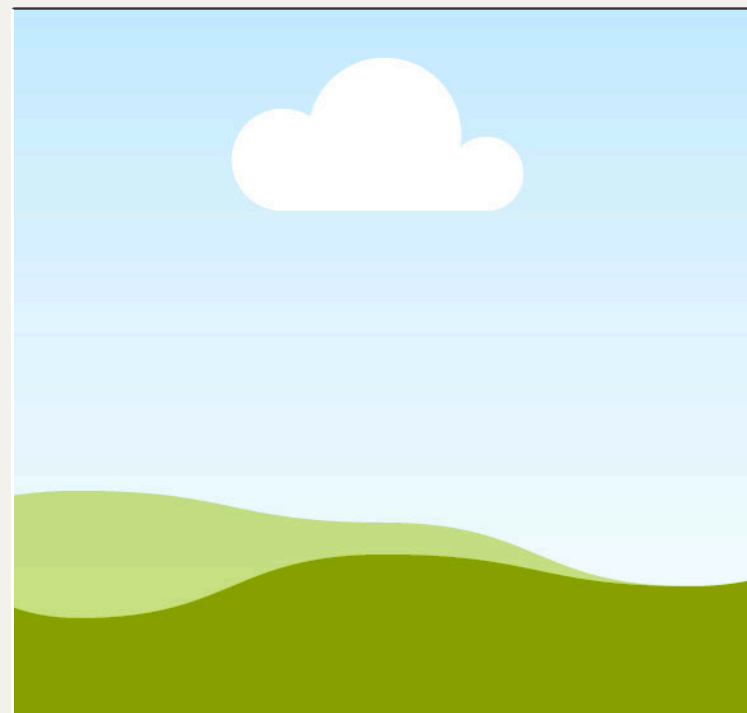
- Oficinas de Robótica
- Oficinas de Programação
- Divulgação do universo da Computação



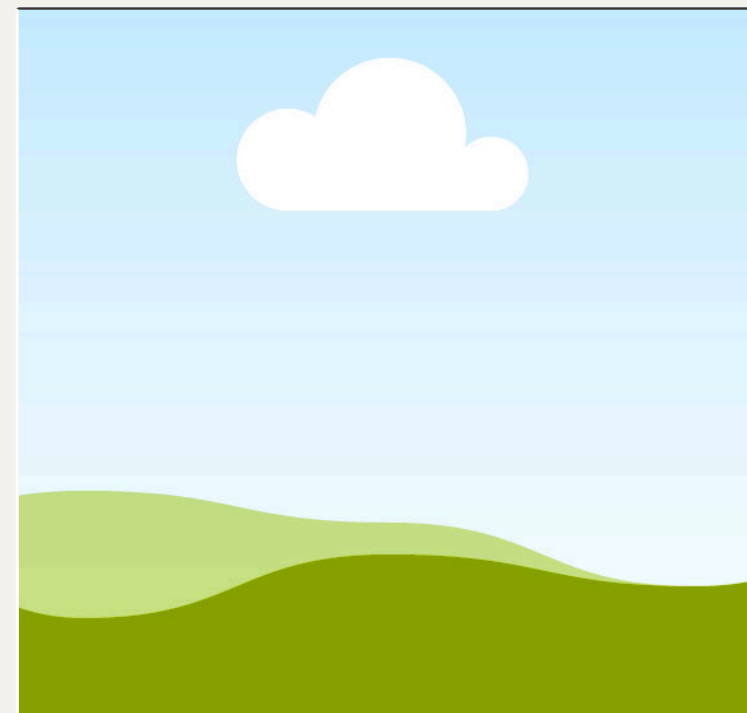
MONITORAS DA OFICINA



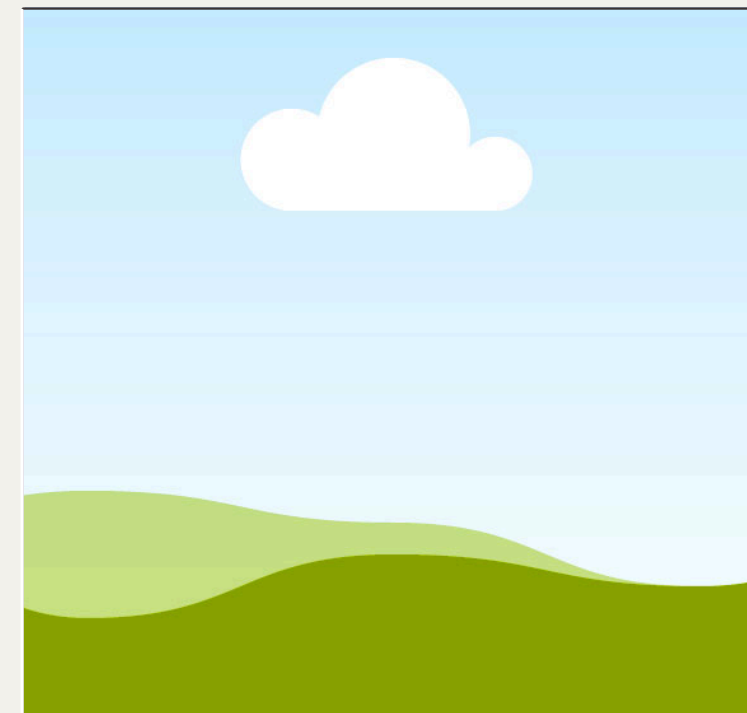
nome1



nome2



nome3



nome4



**VOCÊ JÁ SE
PERGUNTOU COMO A
PROGRAMAÇÃO
PODE SER USADA NO
DIA A DIA?**





NO DIA A DIA

1

Automatizar tarefas

2

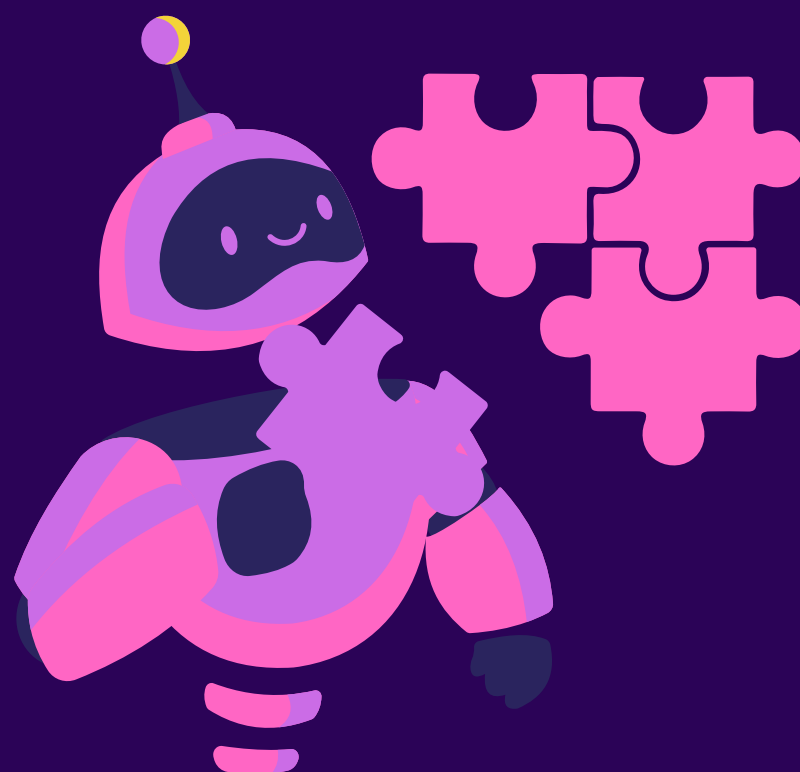
Jogos e diversão

3

Saúde e bem-
estar

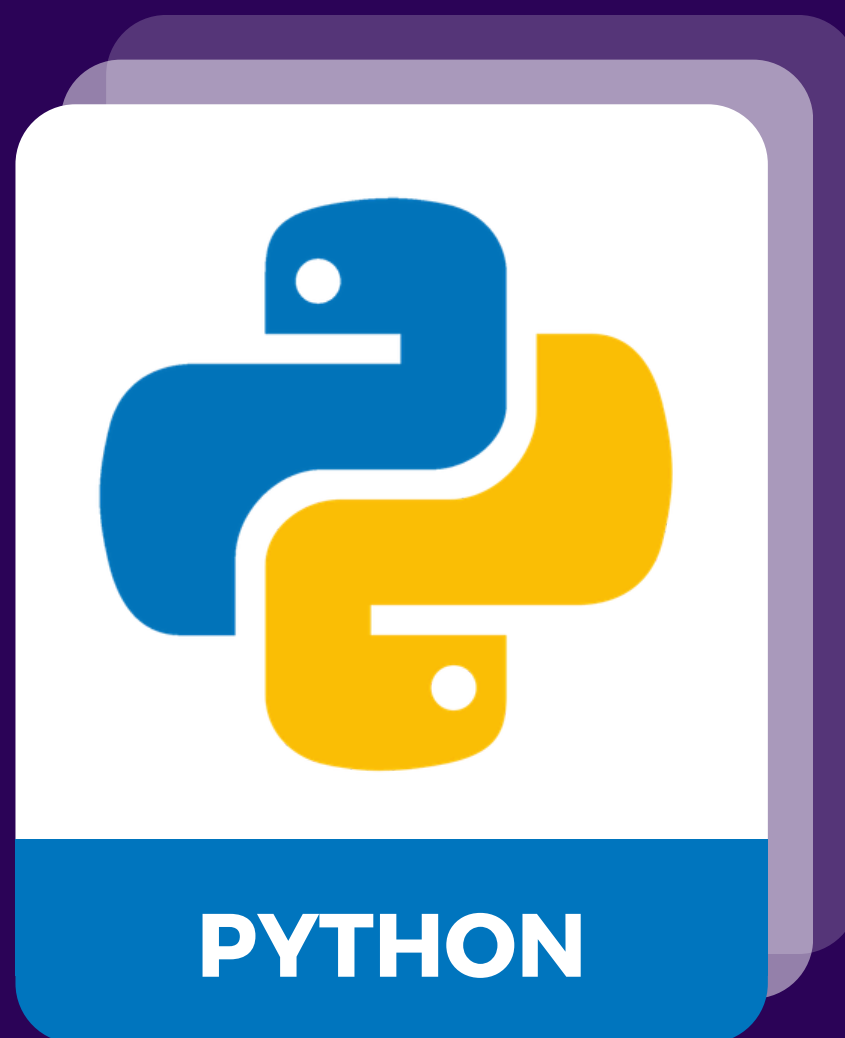


INTRODUÇÃO DAS FERRAMENTAS NECESSÁRIAS PARA A OFICINA EM PYTHON





FERRAMENTAS



Uma linguagem de programação é um conjunto de instruções que usamos para "conversar" com o computador.

Python é uma das linguagens de programação mais usadas no mundo. Ela é conhecida por ser simples, fácil de aprender e muito poderosa.



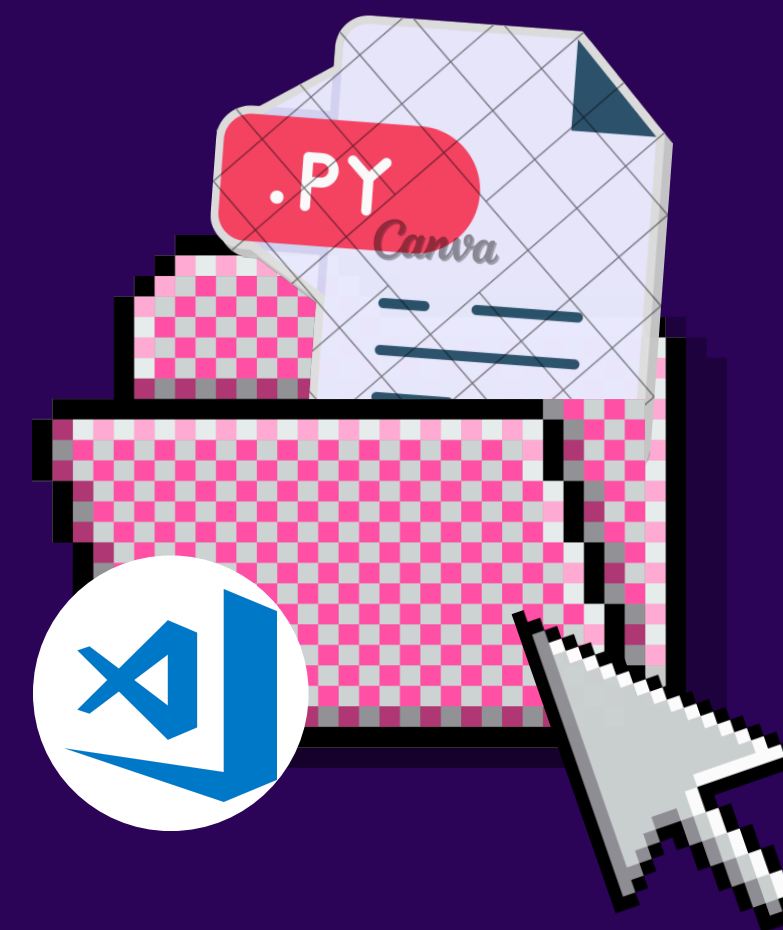
VAMOS COMEÇAR, FUTURAS CODIFICADORAS?





PREPARANDO NOSSO AMBIENTE

1. Acesse o aplicativo **Visual Studio Code**
2. Crie uma pasta para o **nosso projeto**
3. Crie um arquivo **main.py**
4. Crie um arquivo **funcoes_calculadora.py**
5. Crie um arquivo **paleta_de_cores.py**





IMPRIMINDO NA TELA

Exemplo:

```
print("Sejam bem vindas")
```

Python



VARIÁVEL

Podemos **guardar nossos dados** em variáveis!

DADOS

DADOS

DADOS



VARIÁVEL

Tipos de dados que podemos
guardar em uma variável:



NÚMEROS



TEXTOS



**VALORES
BOOLEANOS**



DECLARANDO UMA VARIÁVEL

Exemplo:

Textos sempre devem ser
sinalizados por aspas

```
numero = 20  
nome = "Estefane"  
booleano = True
```

Python



IMPLEMENTANDO O ARQUIVO DE CORES!

No arquivo que criamos `paleta_de_cores.py`

```
#Paleta normal  
preto = "#000000"  
branco = "#FFFFFF"  
cinza = "#3b3b3b"  
cinza_claro = "#D3D3D3"  
cinza_escuro = "#696969"  
prata = "#C0C0C0"
```

```
#Paleta Codificadoras  
roxo_escuro = "#2B0357"  
indigo_escuro = "#2A245E"  
magenta = "#CD6CE6"  
magenta_escuro = "#8B008B"
```



VARIÁVEIS DO TIPO LISTA

Variáveis agrupadas com o mesmo ou diferentes tipo de dados são chamadas de **lista**

List



LISTA

Exemplo:

```
lista = [20, "Kauane", True]  
print(lista[0]) #20
```

Python



Cada elemento da lista tem um índice, que corresponde à sua posição e permite acessar o seu valor



REMOVENDO O ÚLTIMO ELEMENTO DE UMA LISTA

Exemplo:

```
lista = [20, "Kauane", True]    # Python  
  
lista = lista[:-1]  
print(lista)
```

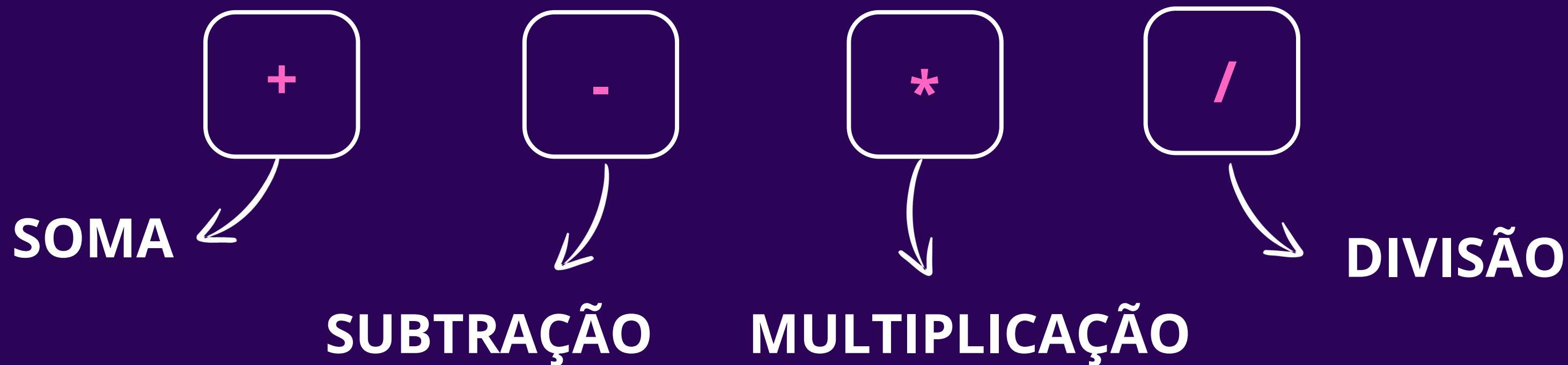
Saída:

```
[20, 'kauane']
```



OPERAÇÕES ARITMÉTICAS

Podemos usar operações aritméticas para fazer **contas matemáticas** em nossos códigos!





OPERADORES LÓGICOS

Principais **operadores lógicos** em Python:

and

E

or

OU

not

NÃO

!=

DIFERENTE



OPERADORES COMPARATIVOS

Principais **operadores comparativos** em
Python:

==

IGUAL

>

MAIOR QUE

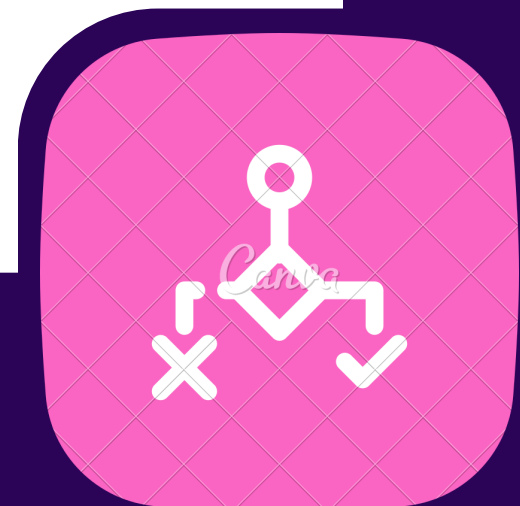
<

MENOR QUE



ESTRUTURAS CONDICIONAIS

Toda vez que queremos que alguma coisa aconteça **se**
uma condição for atendida, utilizamos uma **estrutura**
condicional





ESTRUTURAS CONDICIONAIS: IF/ELSE

Exemplo:

Se minha nota em matemática for maior ou igual a 6, serei aprovada. Se minha nota for menor que 6 e maior que 4 poderei fazer a recuperação, caso contrário, serei reprovada





ESTRUTURAS CONDICIONAIS: IF/ELIF/ELSE

Exemplo:

```
if nota >= 6:                                     # Python
    situacao = "Aprovada"
elif nota >= 4 and nota < 6:
    situacao = "Recuperação"
else:
    situacao = "Reprovada"
```



OPERADORES DE PERTINÊNCIA

Exemplo utilizando in:

```
frutas = ["maçã", "banana", "uva"]           # Python

if "banana" in frutas:
    print("Tem banana na lista!")
```



OPERADORES DE PERTINÊNCIA

Exemplo utilizando not in:

```
itens_bolsa = ["Óculos", "Gloss", "Cartão"] # Python

if "chaves" not in itens_bolsa:
    print("As chaves não estão na bolsa!")
```



ESTRUTURAS CONDICIONAIS: MATCH CASE

Exemplo:

Caso a opção seja 1, fui aprovada, caso a opção seja 2, fui reprovada





ESTRUTURAS CONDICIONAIS: MATCH CASE

Exemplo:

```
opcao = 1
match opcao:
    case 1:
        print("Você foi aprovada!")
    case 2:
        print("Você foi reprovada!")
```

Python



TRY EXCEPT

Exemplo:

```
try:
    numero = int(input("Digite um número: ")) # Python
except ValueError:
    print("A entrada recebida não é um número!")
    exit(1)
```



Utilizamos o try: except para tratar futuros erros do programa!



FUNÇÕES

Podemos criar **funções** para **reaproveitar um bloco de código**



Isso evita repetições de código!



FUNÇÕES

Exemplo:

Bianca quer criar um programa para multiplicar $36 * 7$, $23 * 4$ e $54 * 8$. Como ela sabe que vai realizar a **mesma operação** em todos os casos, Bianca decidiu criar uma **função de multiplicação**.



FUNÇÕES

Exemplo:

```
def multiplicacao(numero1, numero2):           # Python
    conta = numero1 * numero2
    return conta
```



IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def soma(numero1, numero2):  
    resultado = numero1 + numero2  
    return resultado  
  
def subtracao(numero1, numero2):  
    resultado = numero1 - numero2  
    return resultado
```



IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def multiplicacao(numero1, numero2):  
    resultado = numero1 * numero2  
    return resultado
```

```
def divisao(numero1, numero2):  
    resultado = numero1 / numero2  
    return resultado
```



IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def exponenciacao(numero1, numero2):  
    resultado = numero1 ** numero2  
    return resultado
```



VARIÁVEIS GLOBAIS EM FUNÇÕES

Exemplo:

```
contador = 0 #variável global # Python

def aumentar():
    global contador
    contador = contador + 1
    print(f"Contador agora é: {contador}")

aumentar() #1
aumentar() #2
```



INTRODUÇÃO AS BIBLIOTECAS

Exemplo:

```
import math
```

Python

```
def CalculaTangente(numero):  
    conta = math.tan(numero)  
    return conta
```



IMPLEMENTANDO O ARQUIVO MAIN!

No arquivo que criamos main.py

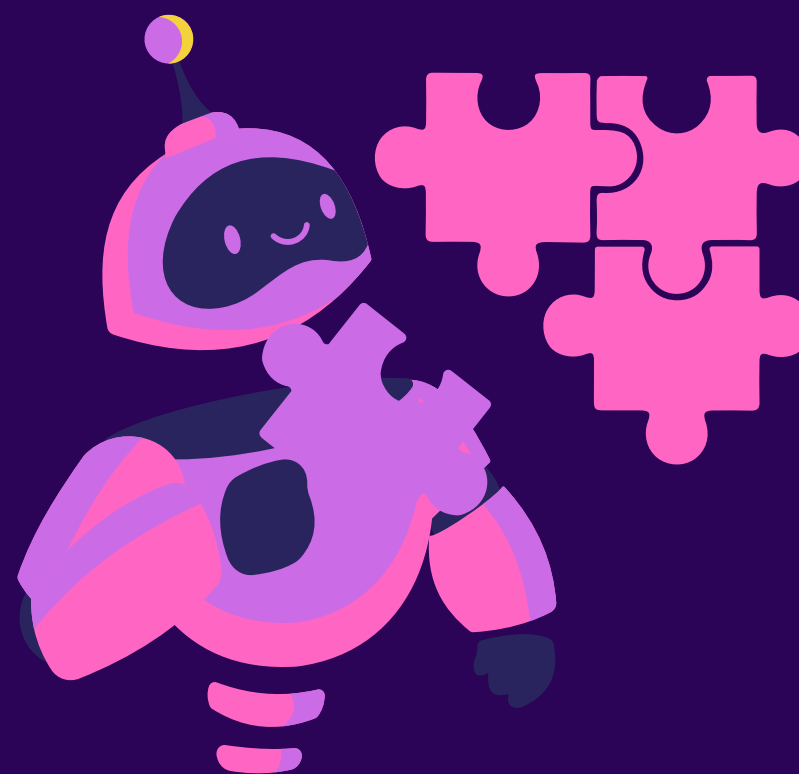
```
import tkinter as tk
from tkinter import *
import tkinter.font as tkFont
import paleta_de_cores as cores
import funcoes_calculadora as funcoes_matematicas
```



Sempre colocamos as bibliotecas nas primeiras linhas do código!



INTRODUÇÃO À BIBLIOTECA TKINTER





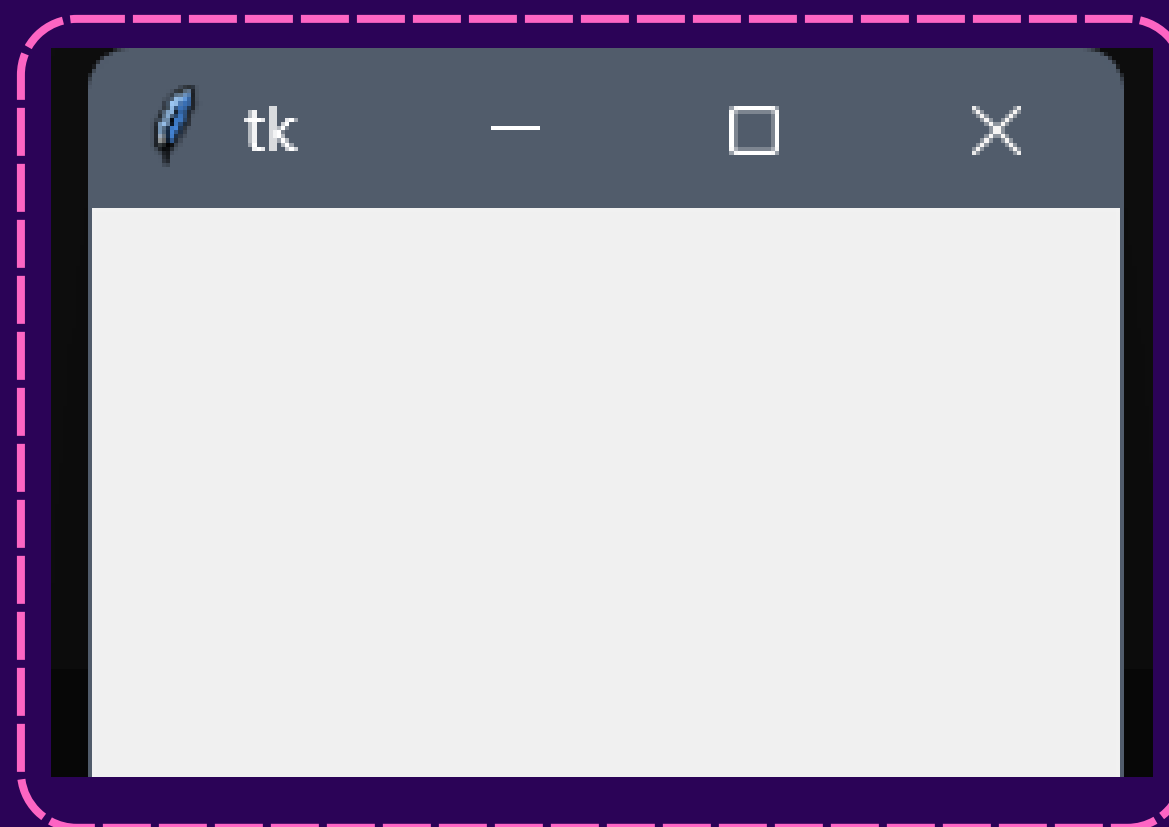
DEFININDO A ABA DA CALCULADORA

Exemplo:

```
janela_teste = tk.Tk() #Define a janela #Python  
  
janela_teste.mainloop() #Executa a página
```



RESULTADO DA ABA DA CALCULADORA





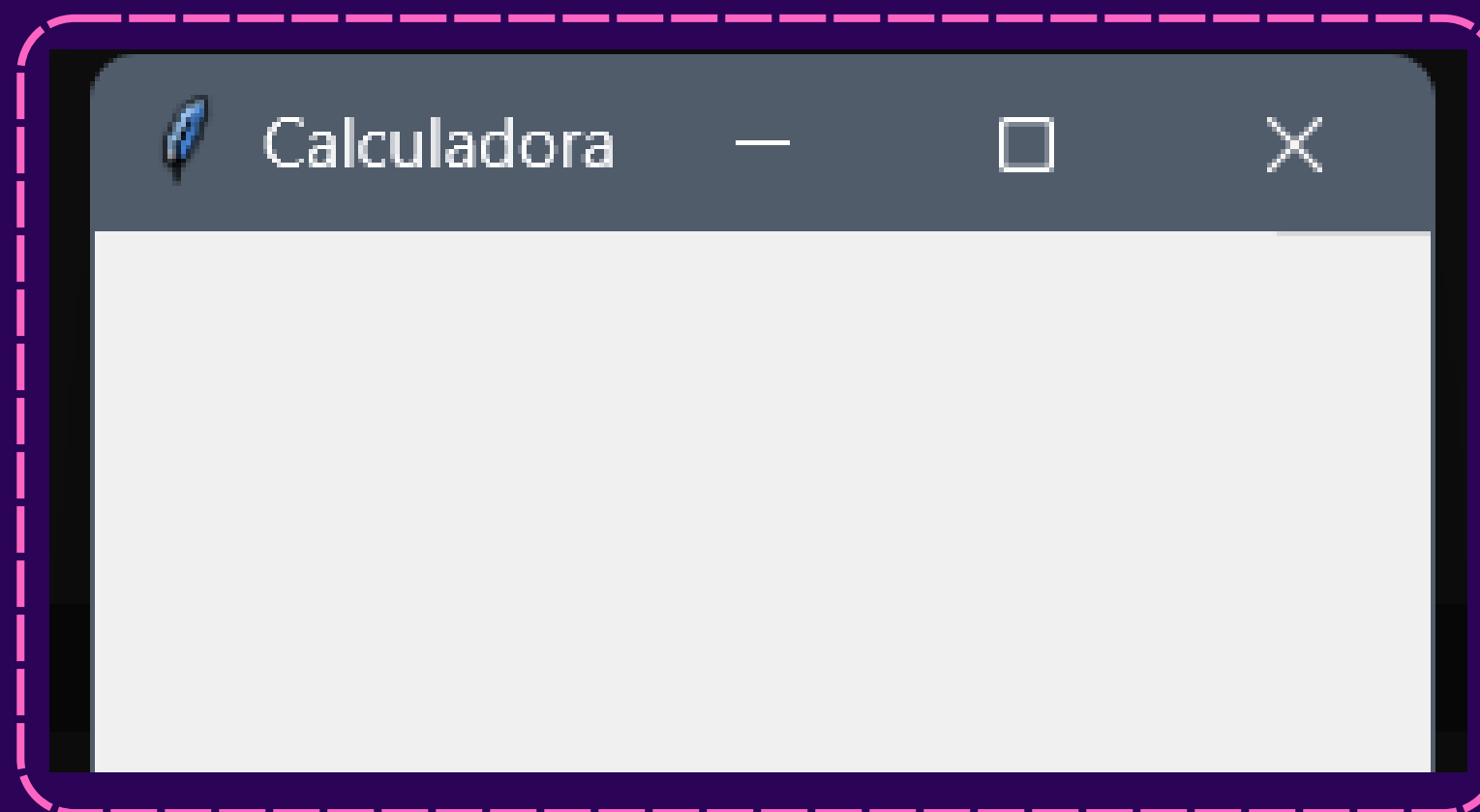
DEFININDO UM TÍTULO NA ABA

Exemplo:

```
janela_teste.title("Calculadora") # Python
```



RESULTADO DO TÍTULO NA ABA





DEFININDO O ÍCONE DA ABA

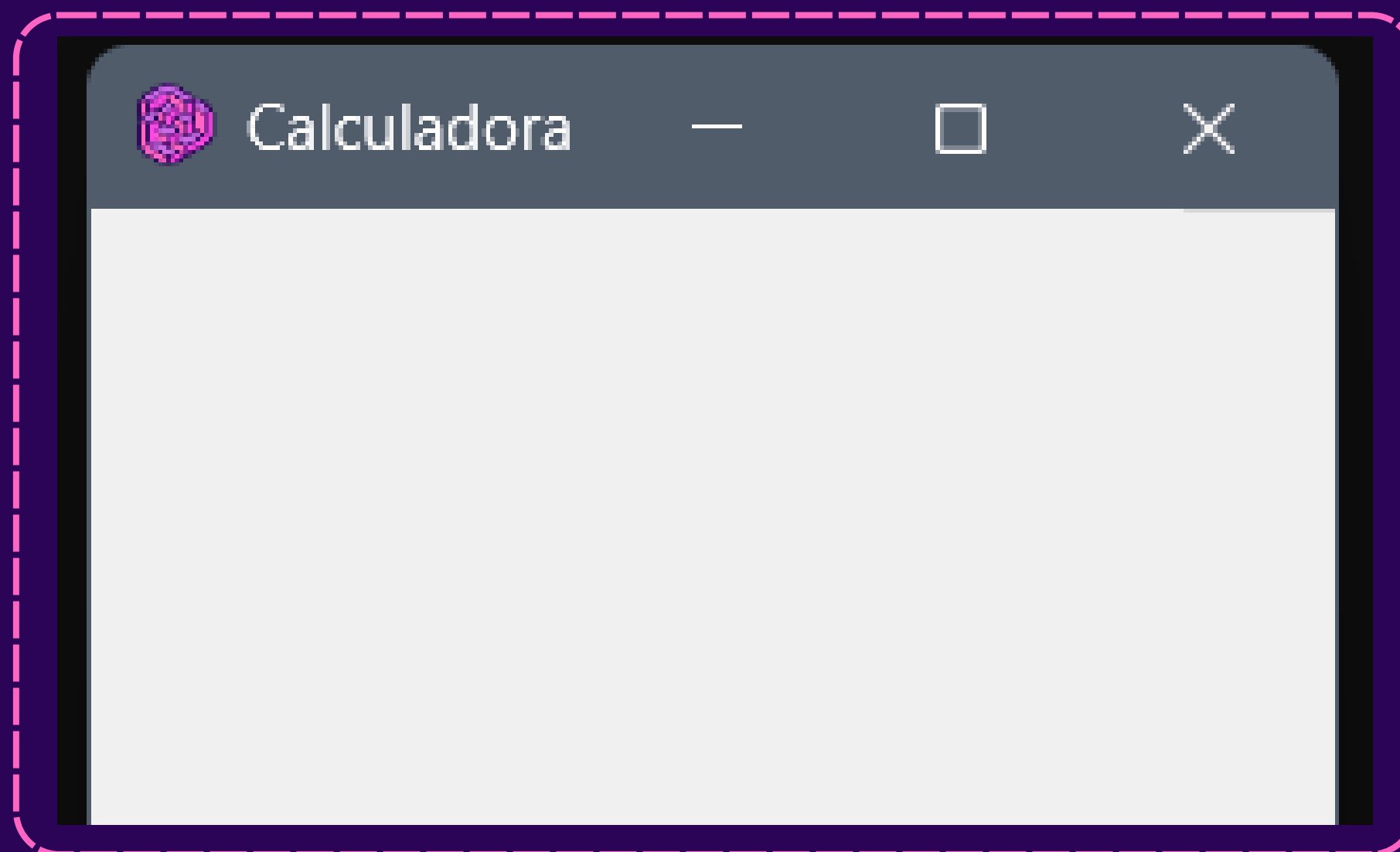
Exemplo:

```
janela_teste.iconbitmap("imagens/logo-com-fundo.ico")
```

Python



RESULTADO DO ÍCONE DA ABA





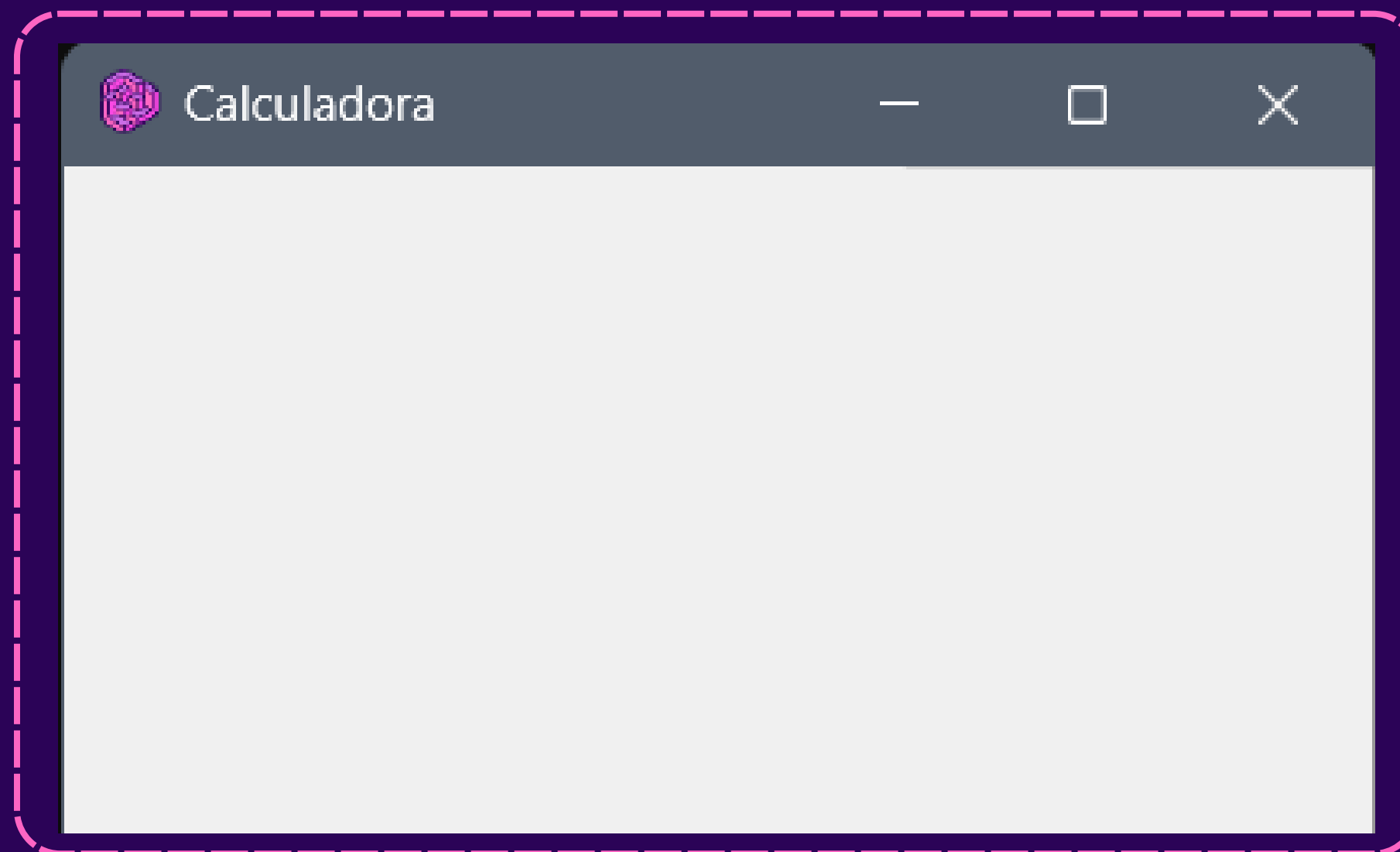
DEFININDO O TAMANHO DA ABA

Exemplo:

```
janela_teste.geometry("276x245") # Python
```



RESULTADO DO TAMANHO DA ABA





TAMANHO E POSIÇÃO DE LAYOUT

`width`

Largura do objeto

`columnspan`

Ocupa várias colunas

`height`

Altura do objeto

`sticky`

Alinhamento

`row`

Posição da linha

`padx`

Espaçamento externo em x

`column`

Posição da coluna

`pady`

Espaçamento externo em y



POSICIONANDO UM OBJETO

Na biblioteca **Tkinter**, há 3 maneiras de se posicionar um objeto

`pack()`

`place()`

`grid()`



POSICIONANDO UM OBJETO: GRID

O grid se baseia em **linhas** e **colunas** para posicionar um objeto

Exemplo:

```
objeto.grid(row=0, column=0) # Python
```



POSICIONANDO UM OBJETO: GRID

Visualizando como o grid funciona

Linha = L	Coluna = C	
L = 0, C = 0	L = 0, C = 1	L = 0, C = 2
L = 1, C = 0	L = 1, C = 1	L = 1, C = 2
L = 2, C = 0	L = 2, C = 1	L = 2, C = 2



PROPRIEDADES DE TEXTO DO LAYOUT

`text`

Exibir o texto

`textvariable`

Texto ligado á variável

`justify`

Alinhamento de texto

`anchor`

Posição de conteúdo

`minsize`

Tamanho mínimo de
linha/coluna



APARÊNCIA DE LAYOUT

`bg`

Cor de fundo do objeto

`font`

Fonte do texto

`fg`

Cor do texto

`relief`

Estilo da borda de botão



DEFININDO UM FRAME NA ABA

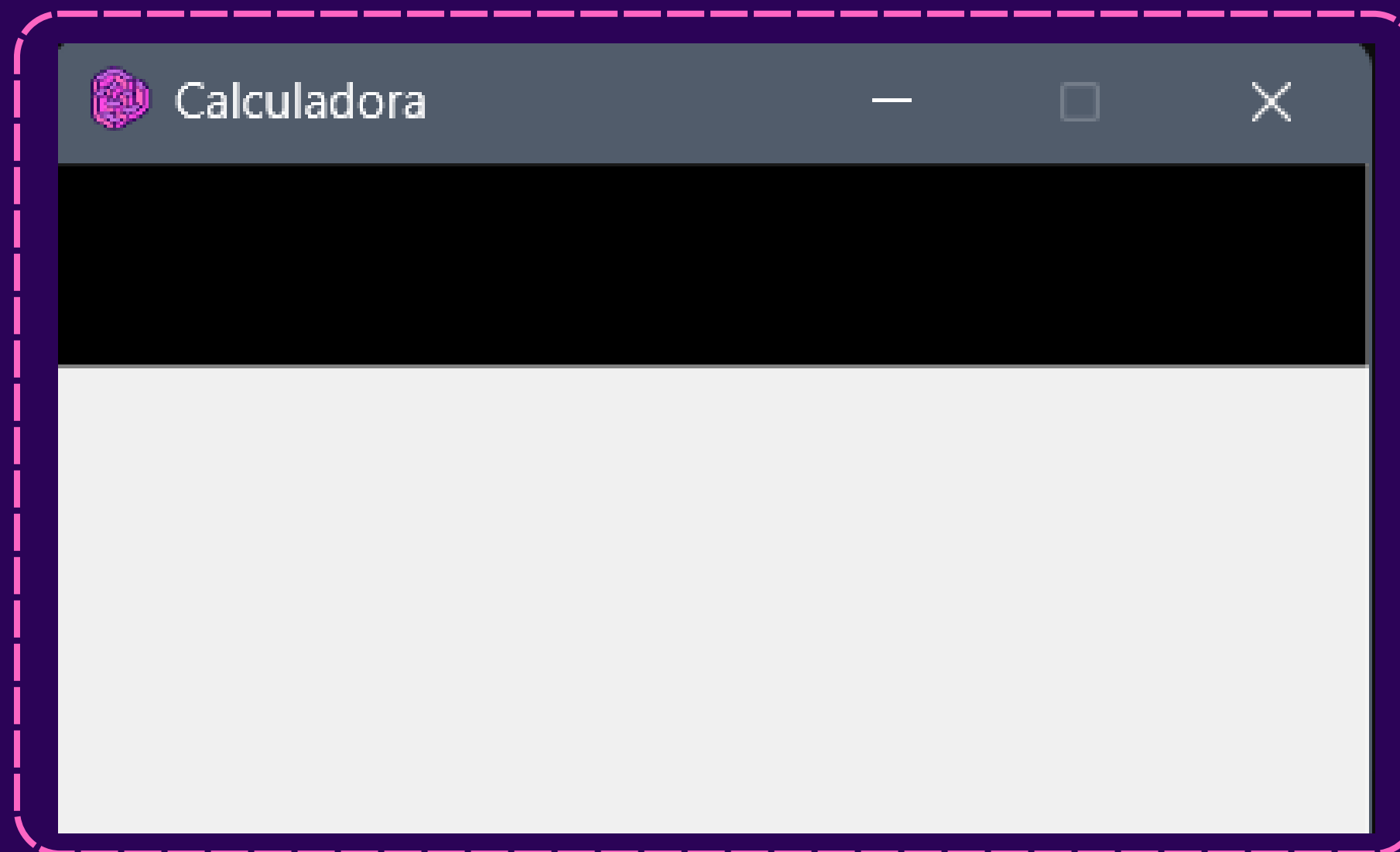
Exemplo:

```
frame_pequeno = Frame(janela_teste, bg="#000000")  
frame_pequeno.grid(row=0, column=0)
```

Python



RESULTADO DO FRAME DA ABA





DEFININDO UM BOTÃO

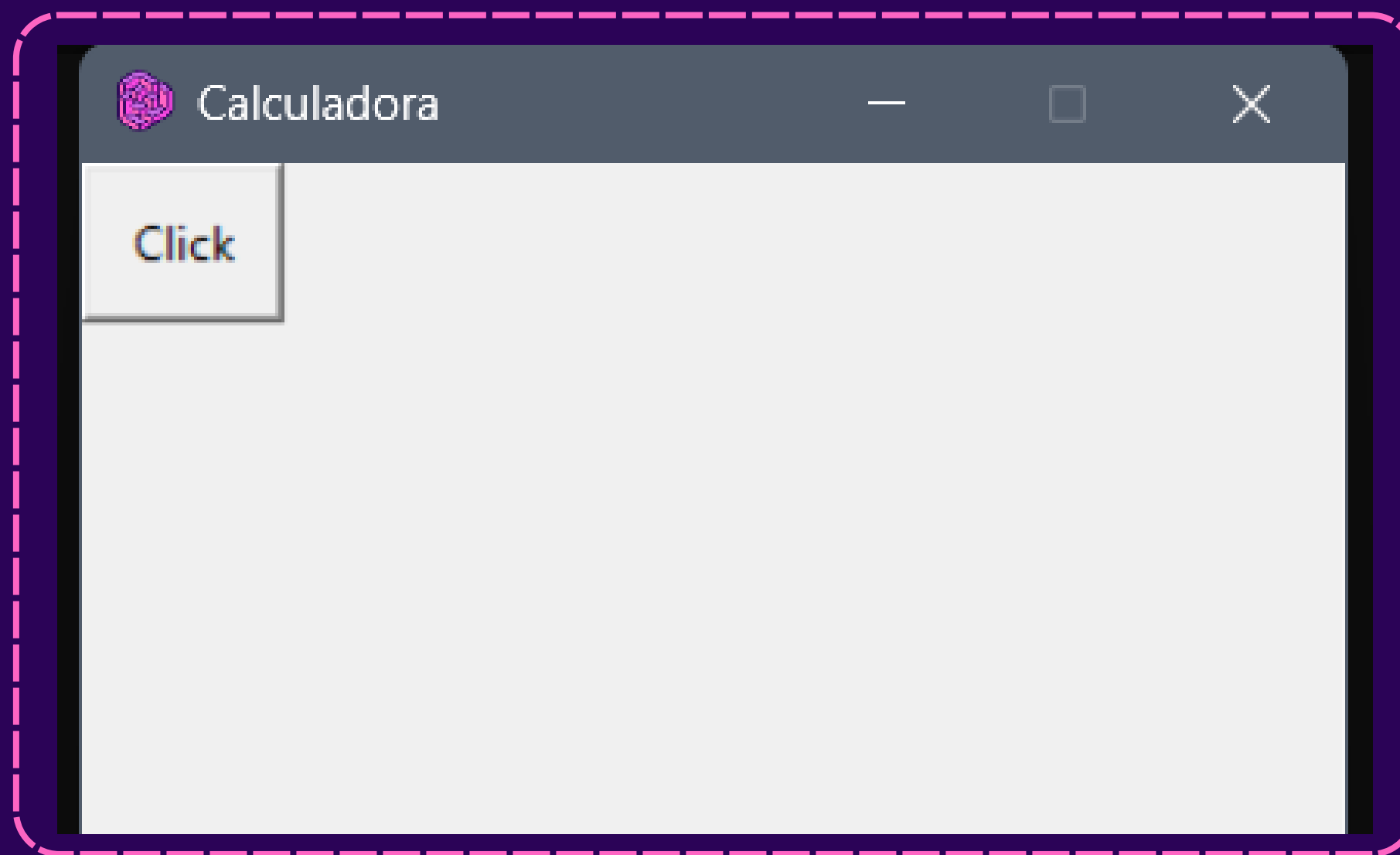
Exemplo:

```
botao = tk.Button(frame_pequeno, text="Click", width=6, height=2, relief="raised")  
botao.grid(row=0, column=0)
```

#Python



RESULTADO DO BOTÃO





CRIANDO EVENTOS DE CLIQUE EM BOTÕES

Exemplo:

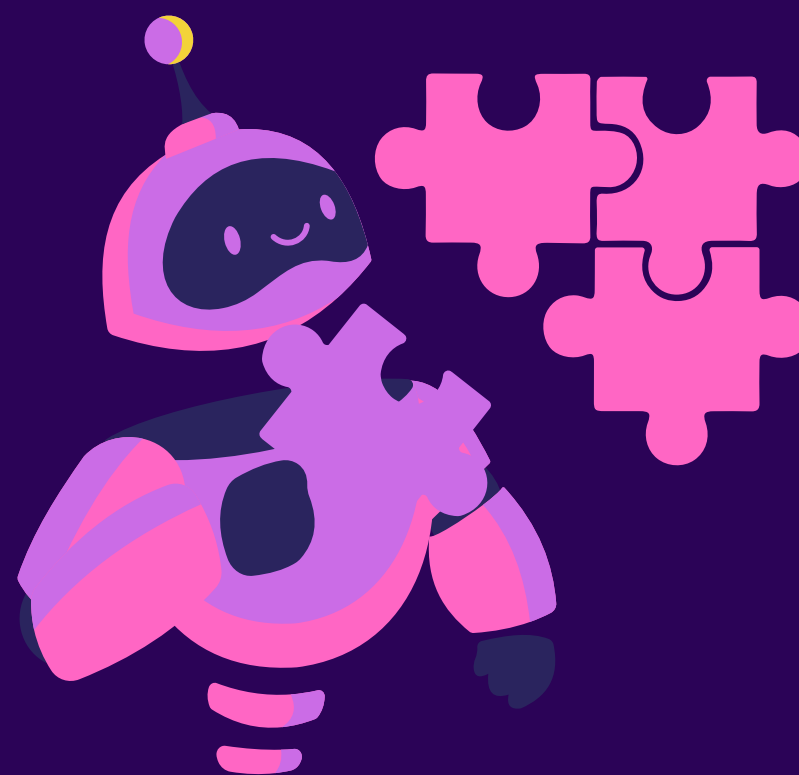
```
def clicar():  
    print("Botao clicado!")
```

#Python

```
botao = tk.Button(frame_pequeno, text="Click", command=clicar, width=6,  
height=2, relief="raised")  
botao.grid(row=0, column=0)
```



**AGORA SABEMOS O
SUFICIENTE PARA CODIFICAR
NOSSA MAIN!**





IMPLEMENTANDO A JANELA DA MAIN!

No arquivo que criamos main.py

```
calculadora = tk.Tk()  
calculadora.title("Calculadora")
```

#Python



IMPLEMENTANDO A JANELA DA MAIN!

No arquivo que criamos `main.py`

```
calculadora.mainloop()
```



Adicione este comando na última linha do nosso código!



IMPLEMENTANDO A JANELA DA MAIN!

No arquivo que criamos `main.py`

```
calculadora.iconbitmap("imagens/logo-com-fundo.ico")  
  
#Define o tamanho da janela  
calculadora.geometry("276x245")  
calculadora.resizable(width=False, height=False)
```



RASCUNHO DA POSIÇÃO DA CALCULADORA

Coluna

Linha

Visor					
		DEL		RESET	ON/OFF
7	8	9		*	/
4	5	6		+	-
1	2	3		X [^]	,
0				=	



Lembrando que o grid de comporta com linhas e colunas!



IMPLEMENTANDO OS FRAMES DA CALCULADORA

No arquivo que criamos `main.py`

```
frame_pequeno = Frame(calculadora, bg=cores.cinza)
frame_pequeno.grid(row=0, column=0)
frame_pequeno.columnconfigure(0, weight=1)
frame_grande = Frame(calculadora)
frame_grande.grid(row=1, column=0, sticky="w")
```



IMPLEMENTANDO OS FRAMES DA CALCULADORA

No arquivo que criamos `main.py`

```
frame_grande.columnconfigure(4, minsize=3)  
frame_grande.rowconfigure(0, minsize=7)  
frame_grande.rowconfigure(2, minsize=7)
```



IMPLEMENTANDO AS VARIÁVEIS DA CALCULADORA

No arquivo que criamos `main.py`

```
valor_texto = StringVar()  
primeiro_numero = None  
operacao = None  
calculadora_ligada = True
```



IMPLEMENTANDO O VISOR CALCULADORA

No arquivo que criamos `main.py`

```
visor_calculadora = Label(frame_pequeno, textvariable=valor_texto, width=22,  
height=2, padx=7, anchor='e', relief=FLAT, justify=RIGHT, font=("Ivy", 16, ""),  
bg=cores.cinza, fg=cores.branco)  
  
visor_calculadora.grid(row=0, column=0)
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_9 = tk.Button(frame_grande, text="9", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_8 = tk.Button(frame_grande, text="8", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_7 = tk.Button(frame_grande, text="7", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_6 = tk.Button(frame_grande, text="6", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_5 = tk.Button(frame_grande, text="5", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_4 = tk.Button(frame_grande, text="4", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_3 = tk.Button(frame_grande, text="3", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_2 = tk.Button(frame_grande, text="2", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_1 = tk.Button(frame_grande, text="1", width=4, height=1, padx=5, pady=5,  
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_0 = tk.Button(frame_grande, text="0", width=4, height=1, padx=4, pady=4,
relief="raised", bg=cores.cinza_claro, fg=cores.preto, font=("Ivy", 10, ""))

botao_soma=tk.Button(frame_grande, text="+", width=4, height=1, padx=3, pady=5,
relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))

botao_subtracao=tk.Button(frame_grande, text="-", width=4, height=1, padx=3, pady=5,
relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_multiplicacao=tk.Button(frame_grande, text="*", width=4, height=1, padx=3, pady=5,  
relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_divisao=tk.Button(frame_grande, text="÷", width=4, height=1, padx=3, pady=5,  
relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))
```

```
botao_exponenciacao=tk.Button(frame_grande, text="x^", width=4, height=1, padx=3, pady=5,  
relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_virgula=tk.Button(frame_grande, text=".", width=4, height=1, padx=3, pady=5, relief="raised",  
    , bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))  
  
botao_igual=tk.Button(frame_grande, text="=", width=4, height=1, padx=3, pady=5, relief="raised",  
    , bg=cores.prata, fg=cores.preto, font=("Ivy", 10, ""))  
  
botao_on_off=tk.Button(frame_grande, text="ON/OFF", relief="raised", bg=cores.magenta,  
    , fg=cores.preto, font=("Ivy", 10, ""))
```



DECLARANDO OS BOTÕES CALCULADORA

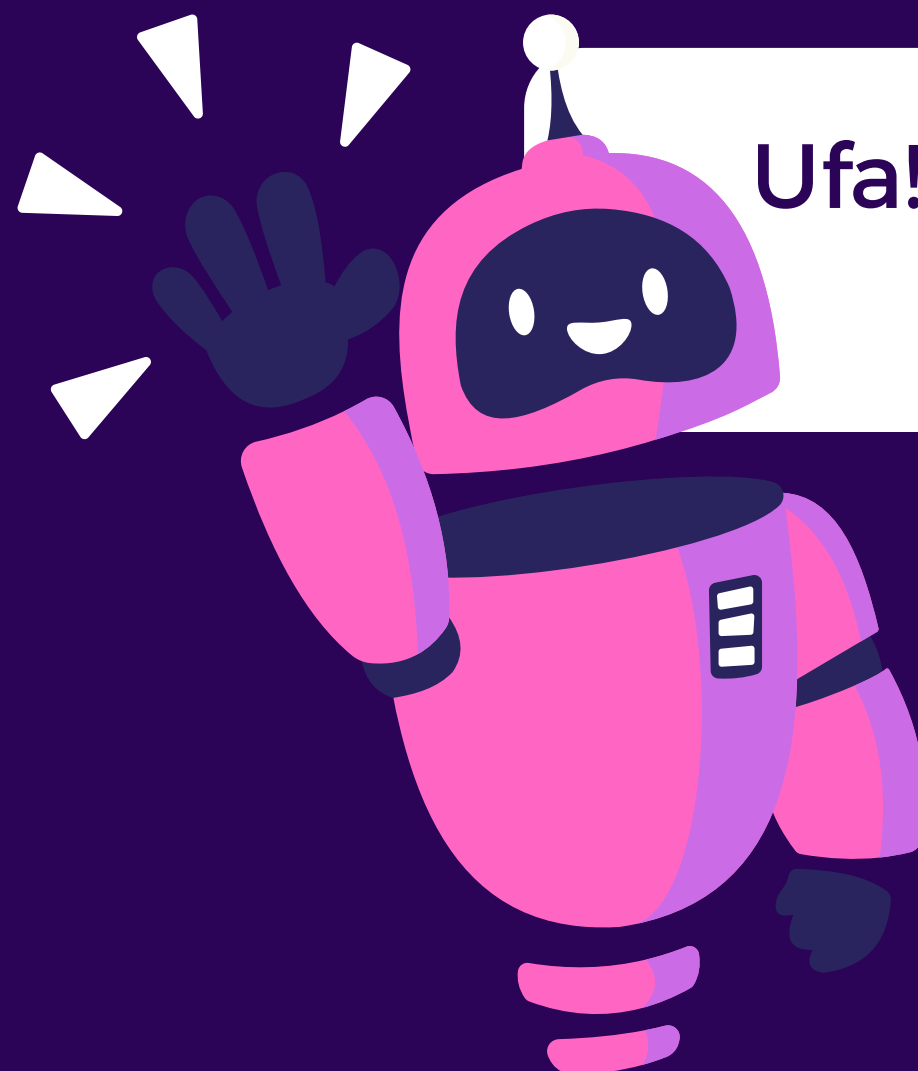
No arquivo que criamos `main.py`

```
botao_del = tk.Button(frame_grande, text="DEL", relief="raised", bg=cores.prata,
fg=cores.preto, font=("Ivy", 10, ""))

botao_reset = tk.Button(frame_grande, text="RESET", relief="raised", bg=cores.prata,
fg=cores.preto, font=("Ivy", 10, ""))
```



IMPLEMENTANDO O GRID NA CALCULADORA!



Ufa! Botões declarados, agora vamos posicionar eles usando **grid**!



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_9.grid(row=3, column=3, padx=1, pady=1, sticky="nsew")
botao_8.grid(row=3, column=2, padx=1, pady=1, sticky="nsew")
botao_7.grid(row=3, column=1, padx=1, pady=1, sticky="nsew")
botao_6.grid(row=4, column=3, padx=1, pady=1, sticky="nsew")
botao_5.grid(row=4, column=2, padx=1, pady=1, sticky="nsew")
botao_4.grid(row=4, column=1, padx=1, pady=1, sticky="nsew")
botao_3.grid(row=5, column=3, padx=1, pady=1, sticky="nsew")
```



DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_2.grid(row=5, column=2, padx=1, pady=1, sticky="nsew")
botao_1.grid(row=5, column=1, padx=1, pady=1, sticky="nsew")
botao_0.grid(row=6, column=1, colspan=3, sticky="we", padx=1, pady=1)
botao_soma.grid(row=4, column=5, padx=1, pady=1, sticky="nsew")
botao_subtracao.grid(row=4, column=6, padx=1, pady=1, sticky="nsew")
botao_multiplicacao.grid(row=3, column=5, padx=1, pady=1, sticky="nsew")
botao_divisao.grid(row=3, column=6, padx=1, pady=1, sticky="nsew")
```



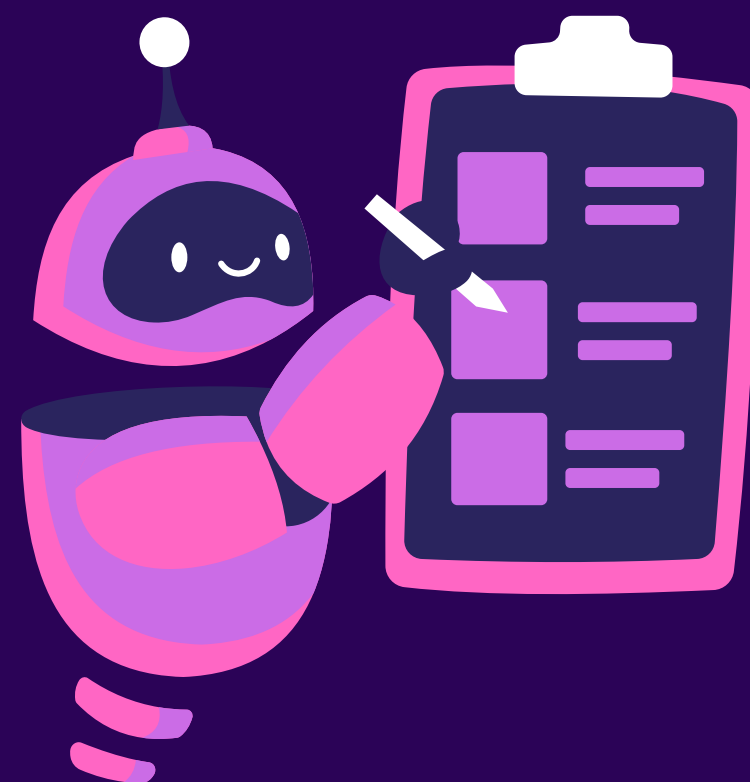
DECLARANDO OS BOTÕES CALCULADORA

No arquivo que criamos `main.py`

```
botao_exponenciacao.grid(row=5, column=5, padx=1, pady=1, sticky="nsew")
botao_igual.grid(row=6, column=5, padx=1, pady=1, columnspan=2, sticky="nsew")
botao_virgula.grid(row=5, column=6, padx=1, pady=1, sticky="nsew")
botao_on_off.grid(row=1, column=6)
botao_reset.grid(row=1, column=5)
botao_del.grid(row=1, column=3, sticky="nsew")
```



AGORA NÓS PODEMOS FAZER NOSSAS FUNÇÕES DA CALCULADORA!





IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

```
def inserir_valor(valor):  
    global valor_texto  
    atual = valor_texto.get()  
    valor_texto.set(atual + str(valor))
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

```
def inserir_operacao(op):  
    global primeiro_numero, operacao  
  
    if op == "-" and valor_texto.get() == "":  
        valor_texto.set("-")  
        return  
  
    elif op == "-" and valor_texto.get() == "-":  
        valor_texto.set("+")  
        operacao = "+"  
        return  
  
    primeiro_numero = float(valor_texto.get())  
    operacao = op  
    valor_texto.set("")
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos main.py

```
def inserir_virgula():  
    atual = valor_texto.get()  
  
    if "." not in atual:  
        if atual == "":  
            valor_texto.set("0.")  
        else:  
            valor_texto.set(atual + ".")
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

```
#Função que vai ser usada na configuração do botão de l  
def deletar_caractere():  
    atual = valor_texto.get()  
    valor_texto.set(atual[:-1])
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

```
def resetar():  
    global primeiro_numero, operacao  
    valor_texto.set("")  
    primeiro_numero = None  
    operacao = None
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos main.py
Parte 1

```
def calcular():  
    global primeiro_numero, operacao  
  
    try:  
        segundo_numero = float(valor_texto.get())  
    except ValueError:  
        valor_texto.set("Erro: insira um número")  
    return
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

Parte 2

#Ainda na função calcular

`match` operacao:

`case "+"`:

`resultado = funcoes_matematicas.soma(primeiro_numero, segundo_numero)`

`case "-"`:

`resultado = funcoes_matematicas.subtracao(primeiro_numero, segundo_numero)`

`case "*"`:

`resultado = funcoes_matematicas.multiplicacao(primeiro_numero, segundo_numero)`



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos main.py

Parte 3

#Ainda na função calcular

```
case "÷":
```

```
    if segundo_numero != 0:
```

```
        resultado = funcoes_matematicas.divisao(primeiro_numero, segundo_numero)
```

```
    else:
```

```
        valor_texto.set("Indefinido!")
```

```
    return
```

```
case "x^":
```

```
    resultado = funcoes_matematicas.exponenciacao(primeiro_numero, segundo_numero)
```



IMPLEMENTANDO AS FUNÇÕES DA CALCULADORA!

No arquivo que criamos `main.py`

Parte 4

```
#Ainda na função calcular  
case _:  
    valor_texto.set(str("Operações unárias não precisam do '='"))  
    return  
  
valor_texto.set(str(resultado))
```



IMPLEMENTANDO A FUNÇÃO DE LIGAR/DESLIGAR

Para fazermos a função para **ligar/desligar** a calculadora, nós precisamos **habilitar** e **desabilitar todos** os **botões**, mas como podemos fazer isso?





PASSO 1: LISTA DE BOTÕES!

Passo 1: Primeiro precisamos fazer uma **lista de botões**, ou seja, vamos fazer uma lista de todos os botões que temos no programa!





RESULTADO DA LISTA:

```
lista_botoes = [botao_0, botao_1, botao_2, botao_3, botao_4, botao_5, botao_6,  
botao_7, botao_8, botao_9, botao_soma, botao_subtracao, botao_multiplicacao,  
botao_divisao, botao_exponenciacao, botao_del, botao_reset, botao_virgula,  
botao_igual]
```



Sugestão: Coloque essa lista abaixo dos comandos de declaração dos botões



PASSO 2: FUNÇÃO DE LIGAR/DESLIGAR

Passo 2: Agora precisamos fazer uma **função** que **habilite/desabilite** esses botões!





RESULTADO DA FUNÇÃO!

No arquivo que criamos main.py

Parte 1

```
def liga_desliga():  
    global calculadora_ligada  
    calculadora_ligada = not calculadora_ligada  
  
    if calculadora_ligada == True:  
        valor_texto.set("")  
        estado_calculadora = "normal"  
        visor_calculadora.config(anchor='e', justify=RIGHT, font=("Ivy", 16, ""), fg=cores.branco)
```



RESULTADO DA FUNÇÃO!

No arquivo que criamos main.py

Parte 2

```
#Ainda na função liga_desliga()
else:
    visor_calculadora.config(anchor='center', justify=CENTER, font=(tkFont.ITALIC, 13,
"bold"), fg=cores.preto)
    valor_texto.set("-----")
    estado_calculadora = "disable"

    for botao in lista_botoes:
        botao.config(state=estado_calculadora)
```



IMPLEMENTANDO OS EVENTOS NOS BOTÕES!

Tínhamos aprendido a programar **eventos** nos botões nos slides sobre a **tkinter**, agora iremos utilizar o **command** e o **command:lambda** para implementar esses eventos!





IMPLEMENTANDO COMMAND NOS BOTÕES!

No arquivo que criamos `main.py`

```
botao_9.config(command=lambda: inserir_valor(9))  
botao_8.config(command=lambda: inserir_valor(8))  
botao_7.config(command=lambda: inserir_valor(7))  
botao_6.config(command=lambda: inserir_valor(6))  
botao_5.config(command=lambda: inserir_valor(5))  
botao_4.config(command=lambda: inserir_valor(4))
```



IMPLEMENTANDO COMMAND NOS BOTÕES!

No arquivo que criamos `main.py`

```
botao_3.config(command=lambda: inserir_valor(3))  
botao_2.config(command=lambda: inserir_valor(2))  
botao_1.config(command=lambda: inserir_valor(1))  
botao_0.config(command=lambda: inserir_valor(0))  
botao_multiplicacao.config(command=lambda: inserir_operacao("*"))  
botao_exponenciacao.config(command=lambda: inserir_operacao("x^"))  
botao_subtracao.config(command=lambda: inserir_operacao("-"))
```



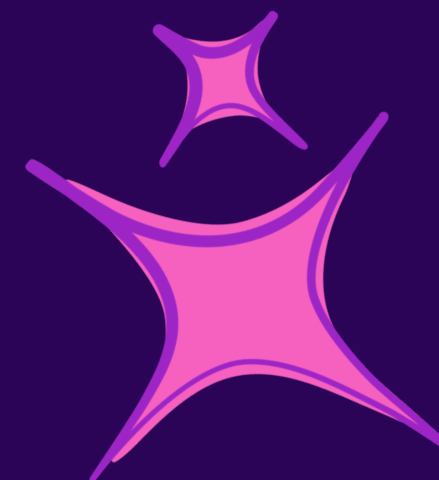
IMPLEMENTANDO COMMAND NOS BOTÕES!

No arquivo que criamos main.py

```
botao_divisao.config(command=lambda: inserir_operacao("÷"))  
botao_soma.config(command=lambda: inserir_operacao("+"))  
botao_igual.config(command=calcular)  
botao_virgula.config(command=inserir_virgula)  
botao_del.config(command=deletar_caractere)  
botao_reset.config(command=resetar)  
botao_on_off.config(command=liga_desliga)
```



RESULTADO ESPERADO:





**NÓS CONSEGUIMOS
FAZER UMA
CALCULADORA COM
INTERFACE DO ZERO!**





PRONTOS PARA A SEGUNDA ETAPA?





PRÓXIMOS PASSOS:

E se além das funções básicas, **fizessemos operações trigonométricas**, e **operações unárias** como raiz quadrada e raiz cúbica? Como poderíamos dividir isso para implementar na nossa calculadora?





PRÓXIMOS PASSOS:

Passo 1: Colocar as novas funções no arquivo de funções

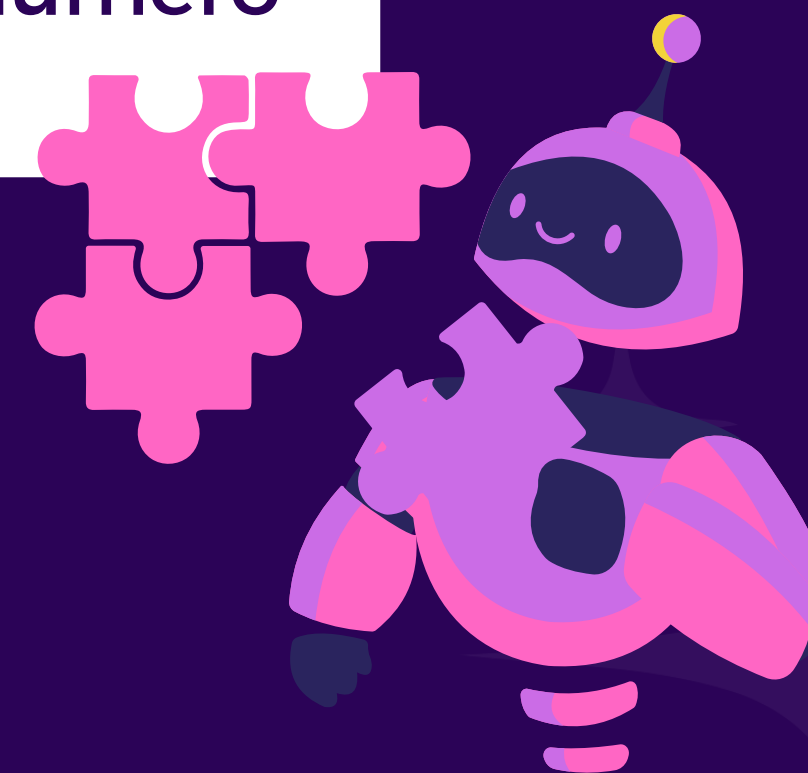
Passo 2: Realocar alguns botões de lugar

Passo 3: Colocar os novos botões na Calculadora

Passo 4: Adicionar esses botões novos na lista de botões

Passo 5: Fazer a função que interliga o botão e o número

Passo 6: Configurar os botões com o command





IMPLEMENTANDO O ARQUIVO DE FUNÇÕES

No arquivo que criamos `funcoes_calculadora.py`

```
import math
```



Esta biblioteca vai ser importante para calcular nossas novas funções



PASSO 1: IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def raiz_cubica(numero):  
    resultado = math.cbrt(numero)  
    return resultado  
  
def raiz_quadrada(numero):  
    resultado = math.sqrt(numero)  
    return resultado
```



PASSO 1: IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def funcao_seno(numero):  
    resultado = math.sin(numero)  
    return resultado  
  
def funcao_cosseno(numero):  
    resultado = math.cos(numero)  
    return resultado
```



PASSO 1: IMPLEMENTANDO O ARQUIVO DE FUNÇÕES!

No arquivo que criamos `funcoes_calculadora.py`

```
def funcao_tangente(numero):  
    resultado = math.tan(numero)  
    return resultado
```



RASCUNHO DA POSIÇÃO DA CALCULADORA

Coluna

Linha

Visor						
				DEL	RESET	ON/OFF
7	8	9		sen(x)	cos(x)	tan(x)
4	5	6		*	/	raiz cúbica
1	2	3		+	-	raiz quadrada
0				=	,	X^



Lembrando que o grid de comporta com linhas e colunas!



PASSO 2: REALOCANDO NOSSOS BOTÕES EXISTENTES

No arquivo que criamos `main.py`

```
botao_del.grid(row=1, column=5, sticky="nsew")  
botao_reset.grid(row=1, column=6)  
botao_on_off.grid(row=1, column=7)  
botao_igual.grid(row=6, column=5, padx=1, pady=1, sticky="nsew")  
botao_virgula.grid(row=6, column=6, padx=1, pady=1, sticky="nsew")
```



PASSO 2: REALOCAR NOSSOS BOTÕES EXISTENTES

No arquivo que criamos `main.py`

```
botao_soma.grid(row=5, column=5, padx=1, pady=1, sticky="nsew")  
botao_subtracao.grid(row=5, column=6, padx=1, pady=1, sticky="nsew")  
botao_multiplicacao.grid(row=4, column=5, padx=1, pady=1, sticky="nsew")  
botao_divisao.grid(row=4, column=6, padx=1, pady=1, sticky="nsew")  
botao_exponenciacao.grid(row=6, column=7, padx=1, pady=1, sticky="nsew")
```



PASSO 3: IMPLEMENTANDO OS NOVOS BOTÕES

No arquivo que criamos main.py

```
botao_raiz_quadrada = tk.Button(frame_grande, text="√", width=4, height=1,  
padx=3, pady=5, relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy",  
10, ""))
```

```
botao_raiz_cubica = tk.Button(frame_grande, text="∛", width=4, height=1,  
padx=3, pady=5, relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy",  
10, ""))
```

```
botao_seno = tk.Button(frame_grande, text="sen(x)", width=4, height=1,  
padx=3, pady=5, relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy",  
10, ""))
```



PASSO 3: IMPLEMENTANDO OS NOVOS BOTÕES

No arquivo que criamos `main.py`

```
botao_cosseno = tk.Button(frame_grande, text="cos(x)", width=4, height=1,  
padx=3, pady=5, relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy",  
10, ""))
```

```
botao_tangente = tk.Button(frame_grande, text="tan(x)", width=4, height=1,  
padx=3, pady=5, relief="raised", bg=cores.prata, fg=cores.preto, font=("Ivy",  
10, ""))
```



PASSO 4: ADICIONANDO OS BOTÕES NA LISTA

No arquivo que criamos `main.py`

```
lista_botoes = [botao_0, botao_1, botao_2, botao_3, botao_4, botao_5,  
botao_6, botao_7, botao_8, botao_9, botao_soma, botao_subtracao,  
botao_multiplicacao, botao_divisao, botao_exponenciacao, botao_raiz_cubica,  
botao_raiz_quadrada, botao_del, botao_reset, botao_virgula, botao_igual,  
botao_seno, botao_cosseno, botao_tangente]
```



PASSO 5: FUNÇÃO QUE RELACIONA A CONTA E O NÚMERO

No arquivo que criamos main.py

```
def calcula_operacoes_unarias(op):  
  
    global operacao  
    operacao = op  
  
    try:  
        numero = float(valor_texto.get())  
        match operacao:  
            case "√":  
                resultado = funcoes_matematicas.raiz_quadrada(numero)  
            case "³√":  
                resultado = funcoes_matematicas.raiz_cubica(numero)  
            case "sen(x)":  
                resultado = funcoes_matematicas.funcao_seno(numero)  
            case "cos(x)":  
                resultado = funcoes_matematicas.funcao_cosseno(numero)
```



PASSO 5: FUNÇÃO QUE RELACIONA A CONTA E O NÚMERO

No arquivo que criamos main.py

```
#Ainda estamos dentro da função
    case "tan(x)":
        resultado = funcoes_matematicas.funcao_tangente(numero)

        valor_texto.set(str(resultado))

except:
    valor_texto.set("Erro.")
```



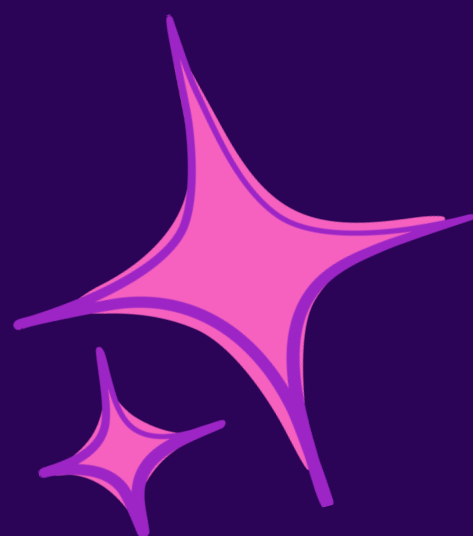
PASSO 6: IMPLEMENTANDO O COMMAND NOS BOTÕES

No arquivo que criamos main.py

```
botao_tangente.config(command=lambda: calcula_operacoes_unarias("tan(x)"))  
botao_raiz_quadrada.config(command=lambda: calcula_operacoes_unarias("√"))  
botao_cosseno.config(command=lambda: calcula_operacoes_unarias("cos(x)"))  
botao_raiz_cubica.config(command=lambda: calcula_operacoes_unarias("³√"))  
botao_seno.config(command=lambda: calcula_operacoes_unarias("sen(x)"))
```



RESULTADO ESPERADO:





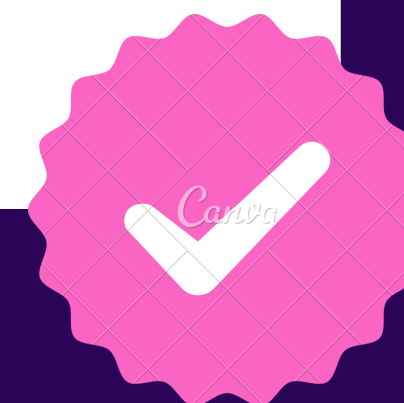
NÓS CONSEGUIMOS FAZER UMA CALCULADORA COM INTERFACE DO ZERO!





RECOMENDAÇÕES

- site **W3Schools**, onde é possível acessar a documentação Python;
- canal do Youtube do **prof. Guanabara**;
- cursos **Udemy**.



Continue estudando!



@CODIFICADORAS.UTFPR
Mulheres que **codificam** o mundo!





FORMULÁRIO DE FEEDBACK

